# CS336 Assignment 3 (scaling): Scaling Laws

Version 26.0.5

CS336 Staff

Spring 2026

## 1 Assignment Overview

In this assignment, you will gain some hands-on experience with language model scaling laws.

**The setting**

You are in charge of training ClosedAI's next-generation language model. This model will take the GDP of a small country to train and melt an Antarctic ice shelf, so you want to make sure you get it right. Concretely, you are given a fixed compute budget (measured in wall-clock time). Your goal is to produce a model with the lowest training loss (i.e., a *compute-optimal* model). To achieve this goal, you will need to figure out how to best trade off (1) training a larger model versus (2) training on more tokens.

Scaling laws, which empirically relate language model training loss to model size and the amount of compute used for training, are frequently used to project this trade-off [J. Kaplan et al., 2020; J. Hoffmann et al., 2022]. In this assignment, you will construct scaling laws to estimate the compute-optimal model size and its corresponding hyperparameters for a model trained on 48 B200 hours. Rather than training the models yourself, you will query a training API with (1) model hyperparameters (i.e., number of Transformer layers, embedding size, number of heads, batch size, learning rate, training tokens, …) and (2) a maximum wall-clock time for the experiment; the API will return the validation loss from training on the corresponding hyperparameters. The budget for fitting our scaling laws is an additional 12 B200 hours (25% of our big run FLOPs budget). The training API accepts a wide range of hyperparameter values, and it is your part of your job to figure out what parts of the search space to explore in order to effectively use the scaling laws budget.

**What the code looks like**

This writeup is available on GitHub at:

`github.com/stanford-cs336/assignment3-scaling`

Please clone the repository using Git. If there are any updates, we will notify you and you can `git pull` to get the latest.

**How to submit**

You will submit the following files to Gradescope:

- `writeup.pdf`: A complete description of your methodology for fitting a scaling law and using it to predict the optimal model size for the given FLOPs budget. The write-up should be detailed enough to reproduce your results.
- `code.zip`: Contains all the code you've written to fit your scaling law and compute your estimates.

In addition, submit your predicted optimal hyperparameters and predicted final validation loss to the api.

Part of your grade on the assignment will be determined by the performance of your predicted optimal model.

**Statement on AI tools**

AI can solve many parts of the assignments fully autonomously. This makes it harder to deeply engage with, and learn from, the course material.

The use of AI tools is permitted for answering high-level conceptual questions, or for providing low-level programming documentation like function signatures and library APIs. However, AI tools are not permitted for implementing any part of any assignment. This includes both coding agents (e.g., Cursor Agents, Codex, Claude Code) and AI autocomplete (e.g., Cursor Tab, GitHub Copilot). When using an AI agent, make sure it uses the AGENTS.md file provided. The prompt should also be included when using chatbots.

We strongly encourage you to disable AI autocomplete (e.g., Cursor Tab, GitHub Copilot) in your IDE when completing assignments (though non-AI autocomplete, e.g., autocompleting function names is totally fine). Previous students have highlighted that disabling AI autocomplete made it easier to engage deeply with the material.

For the full AI Policy, see this document.

# 2 Scaling Laws Review

We will review one of the approaches for fitting scaling laws from the Chinchilla paper [J. Hoffmann et al., 2022]. The central question is: given a compute budget $C$, which we will use to train a large language model, which choice of hyperparameters (model size, number of training tokens, etc.) will lead to the lowest training loss? The main challenge is how to extrapolate from experiments done at a smaller scale to larger scale. For your own work in the second part of the assignment, you are welcome to incorporate ideas from other references, such as J. Kaplan et al. [1] and G. Yang et al. [3].

## 2.1 Scaling Laws from IsoFLOPs profiles

Recall that the compute budget for training a Transformer with $N$ parameters on a dataset of $D$ tokens is approximately $C = 6ND$. The IsoFLOPs approach to scaling laws in J. Hoffmann et al. [2] works as follows: for each compute budget $C$, train language models of varying sizes $N$ given compute budget $C$ (with data size $D = \frac{C}{6N}$), producing a final training loss $L$.

This produces a set of runs that used the same number of FLOPs $C_i$, but have varying model sizes $N_{ij}$. Empirically, J. Hoffmann et al. [2] observe a quadratic relationship between the final training loss $L_{ij}$ and model size $N_{ij}$ given a fixed compute budget $C_i$. One intuition for this is as follows: When $N_i$ is extremely small, the model can't fit the data regardless of how much compute we spend, so the final training loss in this regime is high. As we increase model size, the final training loss drops smoothly, until a certain point where our model starts to be too large and we can't afford to take enough gradient steps within $C$ FLOPs to train it effectively (as an extreme example, as $N_i \to \infty$, even taking a single gradient step would exceed our compute budget $C$, and training would thus stop at very high loss). The method for finding scaling laws will consist of determining the optimal model and dataset sizes for each $C_i$, then fitting a power law that predicts $N$ and $D$ given a target $C$, often a larger budget than what was used in the previous runs.

To fit the power laws, we use data points given by finding the model size $N_{\text{opt}(C_i)}$ that minimizes the training loss for each budget $C_i$, using the set of runs that used that budget (its "IsoFLOPs profile"). This procedure gives us a sequence of pairs $\langle C_i, N_{\text{opt}(C_i)} \rangle$ (and an analogous one for $D_{\text{opt}}$). We use these data points to fit power laws $N_{\text{opt}} \propto C^a$ and $D_{\text{opt}} \propto C^b$, and use these to extrapolate the compute-optimal model and dataset sizes to our target compute budget.

**Problem (`chinchilla_isoflops`): IsoFLOPs scaling laws (5 points)**

Write a script to reproduce the IsoFLOPs method described above for fitting scaling laws using the final training loss from a set of training runs. For this problem, use the (synthetic) data from training runs given in the file `data/isoflops_curves.json`. This file contains a JSON array, where each element is an object describing a training run. Here are the first two runs for illustrating the format:

```
[
    {
        "parameters": 49999999,
        "compute_budget": 6e+18,
        "final_loss": 7.192784500319437
    },
    {
        "parameters": 78730505,
        "compute_budget": 6e+18,
        "final_loss": 6.750171320661809
    },
  ...
]
```

For fitting the scaling laws, the `scipy` package (and `scipy.optimize.curve_fit` in particular) might be useful, but you're welcome to use any curve fitting method you'd like. While J. Hoffmann et al. [2] fits a quadratic function to each IsoFLOP profile to find its minimum, we instead recommend you simply take the run with the lowest training loss for each compute budget as the minimum.

(a) Show your extrapolated compute-optimal model size, together with the $\langle C_i, N_{\text{opt}(C_i)} \rangle$ points you obtained. What is your predicted optimal model size for a budget of $10^{23}$ FLOPs? What about for $10^{24}$ FLOPs?

**Deliverable**: A plot showing your scaling law for model size by compute budget, showing the data points used to fit the scaling law and extrapolating up to at least $10^{24}$ FLOPs. Then, a one-sentence response with your predicted optimal model size.

(b) Show your extrapolated compute-optimal dataset size, together with the $\langle C_i, D_{\text{opt}(C_i)} \rangle$ data points from the training runs. What is your predicted optimal dataset size for budgets of $10^{23}$ and $10^{24}$ FLOPs?

**Deliverable**: A plot showing your scaling law for dataset size by compute budget, showing the data points used to fit the scaling law and extrapolating up to at least $10^{24}$ FLOPs. Then, a one-sentence response with your predicted optimal dataset size.

## 3 Constructing Scaling Laws

In this section, you will use training runs queried through our training API (Section 3.2) under a fixed compute budget to fit a scaling law. Your primary goal is to choose the model size and hyperparameters that will attain the lowest validation loss on 48 B200-hours; you will also predict the validation loss that this model will obtain. To set the hyperparameters for the predicted optimal model size, we recommend analyzing how hyperparameters affect the validation loss for smaller-scale settings. Be sure to carefully plan your runs before you get started–once you exceed the scaling laws budget of 12 B200-hours, the training API will refuse further requests.

### 3.1 Training Run Details

The training API runs real training jobs on B200 GPUs. When you submit an experiment, it enters a shared queue. Queued and running experiments reserve their full `max_runtime_seconds` against your 12-hour scaling-law budget. When an experiment completes or fails, the API recomputes your budget using the actual reported runtime for that experiment, clipped to be at least 1 second and at most `max_runtime_seconds`. Thus, if you reserve 30 minutes for a run that completes in 12 minutes, the unused 18 minutes are returned to your remaining budget. If a run times out, it is charged as `max_runtime_seconds`.

Each training job runs for `n_evals` chunks. The API trains for `total_train_tokens / n_evals` tokens per chunk, evaluates on the validation set after each chunk, and reports the sequence of validation losses in `status.val_losses`. For a completed experiment, the final validation loss is the last entry in this list. For failed timeout experiments, the API may return partial validation losses, but these are not completed runs.

The data order is fixed. The training data is tokenized DCLM data of up to 500B tokens. There is no data epoching. For each training configuration, the API consumes the same deterministic training-token order. The validation set is also using $2^{18}$ validation tokens. The final leaderboard validation loss will use more tokens. The `model_seed` field controls only model initialization, not the data order.

The model architecture largely matches that of assignment 1, with a few slight differences. We provide the code for this model in the assignment repository for your reference. `https://github.com/stanford-cs336/assignment3-scaling`. The training code is only included as a reference. You should only train models through the API.

The models are trained on tokenized DCLM data with a 32K vocabulary. The models' context length is 512. The API supports AdamW and SGD, both with a warmup-cosine learning rate schedule. The API example below uses AdamW with weight decay 0.01, gradient clipping 1.0, warmup fraction 0.05, and final learning rate fraction 0.1. Training uses mean next-token cross-entropy loss over 512-token sequences. There is no dropout in the API model.

### 3.2 Training API

You will use this training API to query the final validation loss for scaling law experiments that you'd like to run. Your API-key will be your 8-digit stanford id (e.g., 06123456) Note that you must be on the Stanford network to query this API, so you may have to use a VPN. As a sanity check, you should be able to see the API documentation page at `http://hyperturing.stanford.edu:8000/docs` and validate that your API key works with the `/budget` endpoint (see below for more details). The training API has the following endpoints:

All API requests should include your API key in an `X-API-Key` header:

```
>>> API_BASE_URL = "http://hyperturing.stanford.edu:8000"
>>> API_KEY = <YOUR_API_KEY_HERE>
>>> headers = {"X-API-Key": API_KEY}
>>> requests.get(f"{API_BASE_URL}/budget", headers=headers).json()
{'used_seconds': 30.0, 'remaining_seconds': 43170.0, 'total_budget_seconds': 43200.0}
```

You will submit experiments to a queue, poll for their status, and read the validation losses once the experiments complete.

**`POST /submit`**

Given a training configuration, queue a training run and return its experiment id. The request body should be a JSON object with the following keys:

- `architecture_config`: A JSON object describing the Transformer architecture. It has the keys `attention_bias`, `head_dim`, `hidden_size`, `intermediate_size`, `num_attention_heads`, `num_hidden_layers`, `num_key_value_heads`, `rms_norm_eps`, `rope_theta`, `tie_word_embeddings`, `dtype`, and `vocab_size`. The `dtype` field must be either `"float32"` or `"bfloat16"`.
- `optimizer_config`: A JSON object describing the optimizer. We provide support for AdamW and SGD configurations; the example below uses AdamW.
- `train_batch_size`: An integer training batch size.
- `val_batch_size`: An integer validation batch size.
- `n_evals`: The number of validation evaluations to run during training.
- `total_train_tokens`: The total number of training tokens.
- `max_runtime_seconds`: The maximum wall-clock runtime to reserve for this experiment. This value must be between 1 second and 12 hours.
- `model_seed`: The random seed used to initialize the model.

The API fixes `seq_len` to 512 and `n_validation_tokens` to $2^{18} \approx 262k$ and validates several consistency conditions. For example, `hidden_size` must equal `num_attention_heads * head_dim`, `num_attention_heads` must be divisible by `num_key_value_heads`, and `total_train_tokens` must be divisible by `512 * train_batch_size`.

The response is a JSON object with the following keys:

- `experiment_id`: An integer id for the queued experiment.
- `budget_summary`: A JSON object with `used_seconds`, `remaining_seconds`, and `total_budget_seconds`.

If you submit the same training configuration twice, the API returns a 409 response and does not reserve additional budget. If `max_runtime_seconds` exceeds your remaining budget, the API returns a 400 response.

To query the API, issue an HTTP POST request:

```
>>> import requests
>>> API_BASE_URL = "http://hyperturing.stanford.edu:8000"
>>> headers = {"X-API-Key": <YOUR_API_KEY_HERE>}
>>> training_config = {
...     "architecture_config": {
...         "attention_bias": False,
...         "head_dim": 64,
...         "hidden_size": 448,
...         "intermediate_size": 1280,
...         "num_attention_heads": 7,
...         "num_hidden_layers": 9,
...         "num_key_value_heads": 7,
...         "rms_norm_eps": 1e-6,
...         "rope_theta": 1_000_000,
...         "tie_word_embeddings": False,
...         "dtype": "bfloat16",
...         "vocab_size": 32_000,
...     },
```

```
...     "optimizer_config": {
...         "lr_scheduler": {
...             "peak_value": 3e-4,
...             "final_lr_frac": 0.1,
...             "warmup_frac": 0.05,
...             "init_value": 0.0,
...         },
...         "weight_decay": 1e-2,
...         "beta1": 0.9,
...         "beta2": 0.95,
...         "eps": 1e-8,
...         "eps_root": 1e-8,
...         "grad_clip_norm": 1.0,
...     },
...     "train_batch_size": 128,
...     "val_batch_size": 32,
...     "n_evals": 16,
...     "total_train_tokens": 1_048_576,
...     "max_runtime_seconds": 30.0,
...     "model_seed": 0,
... }
>>> requests.post(f"{API_BASE_URL}/submit", headers=headers, json=training_config).json()
{'experiment_id': 1, 'budget_summary': {'used_seconds': 30.0, 'remaining_seconds': 43170.0,
'total_budget_seconds': 43200.0}}
```

**`GET /budget`**

Given an API key, returns the wall-clock budget reserved or used by your experiments. The response is a JSON object with the following keys:

- `used_seconds`: The total wall-clock seconds used or reserved by your submitted experiments.
- `remaining_seconds`: The remaining wall-clock seconds in your scaling law budget.
- `total_budget_seconds`: The total wall-clock seconds in the scaling law budget.

To query the API, issue an HTTP GET request:

```
>>> requests.get(f"{API_BASE_URL}/budget", headers=headers).json()
{'used_seconds': 30.0, 'remaining_seconds': 43170.0, 'total_budget_seconds': 43200.0}
```

**`GET /experiments`**

Returns a list of all experiments submitted with your API key. Each experiment has the following keys:

- `experiment_id`: The experiment id returned by `/submit`.
- `training_config`: The training configuration submitted for this experiment.
- `status`: The current experiment status.

The `status.status_type` field is one of:

- `"queued"`: The experiment is waiting to run.
- `"running"`: The experiment is running. The status includes `val_losses`, the validation losses observed so far. The experiment will be in running state after it is queued. This includes startup time and preemption, so your experiment might appear as running for longer than your specified maximum runtime.

- `"completed"`: The experiment finished. The status includes `used_runtime_seconds`, `val_losses`, and `completed_at`.
- `"failed"`: The experiment failed or timed out. Timeout failures include `partial_val_losses`, which you may inspect, but failed experiments have either crashed or timed out.

To query the API, issue an HTTP GET request:

```
>>> requests.get(f"{API_BASE_URL}/experiments", headers=headers).json()
[{'experiment_id': 1, 'training_config': {...}, 'status': {'status_type': 'queued',
'queued_at': '...'}}]
```

**`GET /experiment/{experiment_id}`**

Returns a single experiment with the same response format as the entries returned by `/experiments`. Use this endpoint to poll a submitted experiment until its `status.status_type` is `"completed"`:

```
>>> requests.get(f"{API_BASE_URL}/experiment/1", headers=headers).json()
{'experiment_id': 1, 'training_config': {...}, 'status': {'status_type': 'completed',
'val_losses': [3.1, 2.9, ...], ...}}
```

For completed experiments, use the final entry in `status.val_losses` as the final validation loss for that run.

**`POST /final_submission`**

Submit your predicted optimal hyperparameters and predicted final validation loss. The request body should contain:

- `training_config`: Your predicted optimal training configuration.
- `predicted_final_loss`: Your predicted final validation loss for that configuration.

You may resubmit this endpoint; the latest submission replaces the previous one.

```
>>> final_submission = {
...     "training_config": training_config,
...     "predicted_final_loss": 2.75,
... }
>>> requests.post(f"{API_BASE_URL}/final_submission", headers=headers,
json=final_submission).json()
{'training_config': {...}, 'predicted_final_loss': 2.75, 'submitted_at': '...'}
```

**`GET /final_submission`**

Returns your current final submission, or `None` if you have not submitted one yet:

```
>>> requests.get(f"{API_BASE_URL}/final_submission", headers=headers).json()
{'training_config': {...}, 'predicted_final_loss': 2.75, 'submitted_at': '...'}
```

## 3.3 Leaderboard

You will now use the training API to choose a model configuration for the 48 B200-hour run. The goal is to minimize the validation loss of this final model.

**Problem (`scaling_laws`): Constructing scaling laws leaderboard (50 points)**

Construct a scaling law and use it to choose the model size and hyperparameters that you expect to attain the lowest validation loss on 48 B200-hours. You should also predict the validation loss that this model will obtain. To construct your scaling law, you will use our training API to query the final validation loss for various experimental configurations (Section 3.2); you may not query more than 12 B200-hours worth of experiments for fitting your scaling law. This is a hard cap that will be enforced by the API.

**Deliverable**: A typeset write-up that contains a complete description of your approach and methodology for fitting a scaling law. In addition, it should describe how you use the scaling law to predict the optimal model size for the given FLOPs budget, and your predicted values. The write-up should include commentary about why you made particular design decisions, and the description should be detailed enough to reproduce your approach and results.

We place essentially no constraints on the hyperparameters you may report under the 48 B200-hours run. All runs are on single B200s.

To help you get started, we recommend thinking about at least the following questions. Your writeup should contain additional commentary about how decisions were made for each factor below:

- Given your fixed scaling laws budget of 12 B200-hours, how did you decide which runs to query?
- How did you fit your scaling law? Describe the concrete method or methods you used. In particular, it will likely to be useful to familiarize yourself with the approaches used in J. Kaplan et al. [1] and J. Hoffmann et al. [2].
- How well does your scaling law fit the experimental data?
- For our given FLOPs budget of 48 B200-hours, what optimal model size does your scaling law predict? What is the predicted loss?
- If you were to train a model with your predicted optimal number of parameters, what hyperparameters would you use? To estimate the number of non-embedding parameters for a given model hyperparameter configuration, use $12 n_{\text{layer}} d_{\text{model}}^2$.

In addition to the report, submit your (1) predicted optimal hyperparameters, (2) the model's validation loss to the api. Part of your grade on the assignment will be determined by the performance of your predicted optimal model.

## Bibliography


[1] J. Kaplan *et al.*, "Scaling Laws for Neural Language Models." 2020.

[2] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models." 2022.

[3] G. Yang *et al.*, "Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer." 2022.